

RAG Regression Testing & Operational Evaluation

1. Foundational Context & Evaluation Architecture

1.1 Summary of the RAG Evaluation Framework

- Over the past 5 to 6 sessions, we have thoroughly studied Retrieval Augmented Generation (RAG) evaluations in extensive detail.
 - All of our previous evaluation work can be summarized into a single structural framework: we built a comprehensive **RAG Evaluation Suite** containing a collection of evaluations that allows us to evaluate any RAG application end-to-end.
 - Our evaluation architecture measures system performance across **three distinct structural levels**:
 1. **Component Level**: Evaluates individual isolated modules within the pipeline, such as the retriever or the generator.
 2. **Pipeline Level**: Evaluates the combined interaction between retrieval and generation (e.g., the RAG Triad).
 3. **Application Level**: Evaluates the overall end-to-end user facing behavior of the system.
 - Across these three levels, we have defined and operationalized **14 distinct evaluation metrics**, which break down into four primary categories:
 1. **Quality Metrics**: Includes **Faithfulness**, **Answer Relevancy**, and **Contextual Relevancy**.
 2. **Safety Metrics**: Includes **Toxicity**, **PII Leakage**, and **Scope Adherence**.
 3. **Operational Metrics**: Includes **Latency**, **Cost**, and **Reliability**.
 4. **Retrieval Metrics**: Includes **Recall** and **Precision**.
 - Today's topic focuses on **Regression Testing** specifically how to bundle this entire 14 metric evaluation suite into an automated pipeline using a single master script, execute it, generate performance summaries, and determine whether recent system modifications improved performance or caused silent failures.
 - This session represents the final offline evaluation phase before we move to **Online Evals**, where we will transition these evaluation concepts into production setups to monitor live RAG applications.
-

2. Core Principles of Regression & Regression Testing

2.1 Formal Definition of Regression

- The term **Regression** originates from the standard English definition, *returning to a previous, less advanced, or worse state*.
- In software systems, **regression** occurs when a modification causes the system's performance, quality, safety or operational behavior to degrade compared to its baseline state.
- **Regression Testing** is the explicit process of executing a standardized suite of tests following any system modification to verify whether the system has regressed into a worse state.

2.2 The Single Metric Optimization Trap (Concrete Scenario)

- To understand why regression testing is vital for RAG systems, consider a concrete production scenario:
 1. Suppose we have built a working RAG application, but during analysis, we observe that the **Recall** metric is suboptimal.
 2. To fix the low Recall, we decide to alter our system parameters: we increase the top- k retrieved chunks parameter (e.g., from $k=3$ to $k=5$) and introduce a **Re-ranker** module into the retrieval pipeline.
 3. After implementing these changes, we measure Recall and find it has successfully jumped from **85% to 90%** a clear improvement on paper.
 4. **The Pitfall:** Because we focused exclusively on optimizing Recall, we failed to evaluate the downstream side effects of our changes.
 5. By increasing k and adding a Re-ranker, several critical collateral degradations occurred:
 - **Precision** dropped due to retrieving extra noisy chunks.
 - **Contextual Relevancy** declined because irrelevant context entered the prompt.
 - **Latency** increased significantly due to the additional computational overhead of the Re-ranker.
 6. If we deploy the updated system based solely on the positive Recall jump, we inadvertently push a system into production that severely damages Precision, Contextual Relevancy, and Latency.

[Baseline System] → Modify System (Increase k & Add Re-ranker) → [Candidate System]		
• Recall: 85%		• Recall:
90% (IMPROVED)		
• Precision: High		•
Precision: DOWN (REGRESSED)		
• Context Relevancy: High		• Context
Relevancy: DOWN (REGRESSED)		
• Latency: Low		• Latency:

2.3 The Core Mandate of RAG Regression Testing

- The primary mandate of regression testing is that whenever any change is made to a RAG pipeline (e.g., adjusting \$k\$, tweaking chunk sizes, updating prompt templates, swapping models), we must verify that **all aspects** of the application perform at least as well as they did before the change.
 - We cannot evaluate changes against isolated metrics, we must re run the entire evaluation suite across all 14 metrics, compare the new scores against our established **Baseline**, and confirm that no aspect of the system has regressed prior to deployment.
-

3. Software Engineering Roots & Scope Disclaimers

3.1 Software Engineering Origins

- Regression testing is not a new concept invented for Large Language Model (LLM) applications, it is a long standing foundational methodology in classical software engineering.
- In traditional web applications, Android apps or desktop software, developers routinely perform regression testing.
 - For example, migrating a backend API suite from Flask (V1) to FastAPI (V2) requires running regression suites to confirm that performance, throughput, and endpoint reliability have not degraded.
- This same software engineering discipline applies directly to LLM based application development.

3.2 Pedagogical Scope & Disclaimers

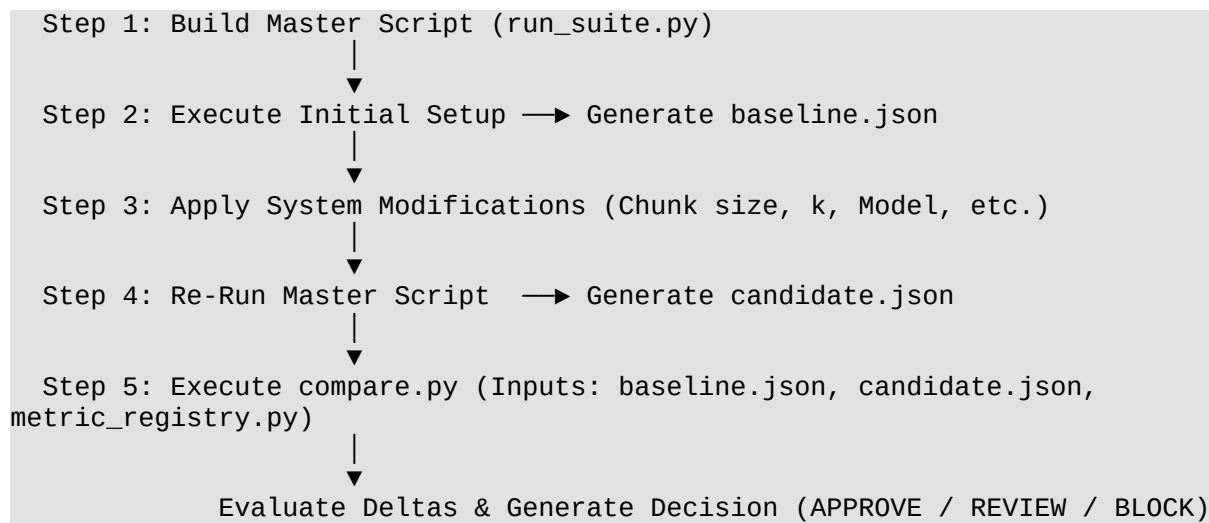
- **Bare Shell Implementation vs. Commercial Tooling:**
 - we implement regression testing using a bare shell method, writing raw custom Python scripts from scratch.
 - In commercial enterprise environments, teams often utilize specialized LLM Ops platforms and tools such as **MLflow** or **Confident AI** to handle regression tracking.
 - We focus on raw code implementation to build a deep, foundational understanding of the underlying theory, statistical logic, and mechanics, which allows seamless adaptation to any enterprise tooling.
- **Golden Dataset Size & Score Variability:**
 - Our evaluation setup uses small **Golden Datasets** (ranging from 5 to 15 rows).
 - Small sample sizes introduce high score variance between identical evaluation runs (e.g., a safety metric like PII Leakage fluctuating between 60% and 80% on consecutive runs

without code changes).

- In production setups, scaling golden datasets to hundreds of representative samples stabilizes metric evaluations significantly.
 - **Session Depth & Intent:**
 - This session provides a practical, high level conceptual overview and local CI/CD workflow for regression testing, focusing on system mechanics rather than inventing new evaluation metrics.
-

4. Architecture of the 5-Step Regression Testing Pipeline

Our evaluation target is the **CampusX Doubt Solver** RAG application. To perform regression testing, we implement a 5-step automated workflow:



Step 1: Build the Master Suite Script (`run_suite.py`)

- Previously, our operational evals, safety evals, and quality evals existed as fragmented, independent Python files executed manually via separate terminal commands.
- We automate this process by constructing a master execution script (`run_suite.py`) that imports and invokes all evaluation modules sequentially, collecting scores across all 14 metrics into a single unified output.

Step 2: Establish the System Baseline (`baseline.json`)

- We execute `run_suite.py` on the unmodified initial state of our RAG application.
- The resulting metric scores across all 14 metrics are saved into a JSON structure named `baseline.json`.
- This file represents our ground truth reference point for all future comparisons.

Step 3: Implement Pipeline Changes

- We introduce targeted modifications to the RAG application.
- Permissible changes include:
 - Modifying vector retrieval settings (e.g., increasing/decreasing top- k).
 - Adjusting text splitter parameters (e.g., chunk size, chunk overlap).
 - Swapping underlying embedding models or generation LLMs.
 - Rewriting system prompt templates.

Step 4: Execute Candidate Run (`candidate.json`)

- We re-run the master execution script (`run_suite.py`) on the modified RAG application.
- The evaluation results across all 14 metrics are captured and saved into a new JSON file named `candidate.json`.

Step 5: Execute Differential Analysis (`compare.py`)

- We run a comparison engine script (`compare.py`) that takes `baseline.json`, `candidate.json`, and our metric metadata registry as inputs.
 - The script calculates the exact **Delta** (Δ) for every metric: $\Delta = \text{Score}_{\text{candidate}} - \text{Score}_{\text{baseline}}$
 - By analyzing the deltas against directionality and statistical noise tolerances, the script categorizes each metric as **IMPROVED**, **FLAT**, or **REGRESSED**.
-

5. Technical Complexities & Solutions

Implementing automated regression testing introduces two major technical complexities that must be explicitly resolved in software:

5.1 Complexity 1: Metric Directionality Asymmetry

- Not all metrics share the same numerical orientation.
- **Higher is Better Metrics:** For quality and retrieval metrics (e.g., **Faithfulness**, **Answer Relevancy**, **Contextual Relevancy**, **Recall**, **Precision**), a higher numerical score signifies better performance. A negative delta $(\Delta < 0)$ indicates performance degradation.
- **Lower is Better Metrics:** For operational and safety metrics (e.g., **Latency**, **Cost**, **Toxicity**, **PII Leakage**), a lower numerical value signifies better performance. A positive delta $(\Delta > 0)$ indicates performance degradation.
- **Solution:** We must encapsulate directionality metadata for every metric inside a centralized

registry.

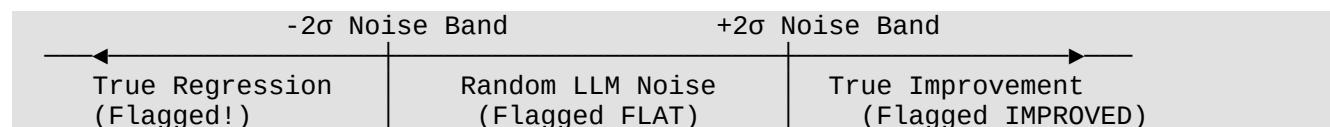
5.2 Complexity 2: Non Deterministic Noise of LLM-as-a-Judge

- Most of our quality and safety evaluation metrics rely on an **LLM-as-a-Judge** framework under the hood.
- Because LLMs are inherently probabilistic text generators, re evaluating an identical, unchanged RAG pipeline across separate runs will produce slight variations in scores.
- *Concrete Example:* If we evaluate Precision on an unmodified pipeline during Run 1, it might score **85.6%**. If we run the exact same evaluation again without making any code changes, Run 2 might score **84.8%**.
- If our comparison script performed naive subtraction, it would falsely flag the drop from 85.6% to 84.8% as a system regression, even though zero code modifications occurred.

5.3 Solution: Statistical Noise Thresholding via 2σ

To prevent false positive regression alerts caused by LLM variance, we implement statistical noise thresholding:

1. **Multi Run Baseline Execution:** Rather than generating `baseline.json` from a single execution, we run the unmodified setup N times (ideally $N = 5$ to 10 runs in production setups) without changing any code.
2. **Standard Deviation Calculation:** For each metric m , we collect N historical scores $(x_1, x_2, \dots, x_N)$ and calculate its sample standard deviation (σ_m) to measure inherent system volatility: $\sigma_m = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$
3. **Noise Threshold Definition:** We define the **Noise Threshold** (or Noise Tolerance) for metric m as **Two Standard Deviations $(2\sigma_m)$** : $\text{Threshold}_m = 2 \cdot \sigma_m$
4. **Classification Logic:**
 - If a measured score change falls within the permissible noise band $[-\text{Threshold}_m, +\text{Threshold}_m]$, the delta is classified as background LLM noise, and the metric status is flagged as **FLAT**.
 - If a score drop exceeds the $2\sigma_m$ threshold, it is classified as true system **Regression**.



- *Numerical Example:* Suppose Recall has a baseline mean of 90.0% with a standard deviation $\sigma = 0.5\%$, yielding a noise threshold $2\sigma = 1.0\%$.
 - If a parameter change drops Recall to 89.5% ($\Delta = -0.5\%$), since $|-0.5\%| \leq 1.0\%$, the change is attributed to LLM noise and marked **FLAT**.

- If Recall drops to 75.5% ($\Delta = -2.5\%$), since $-2.5\% > -1.0\%$, it breaches the noise boundary and is flagged as true **Regression**.

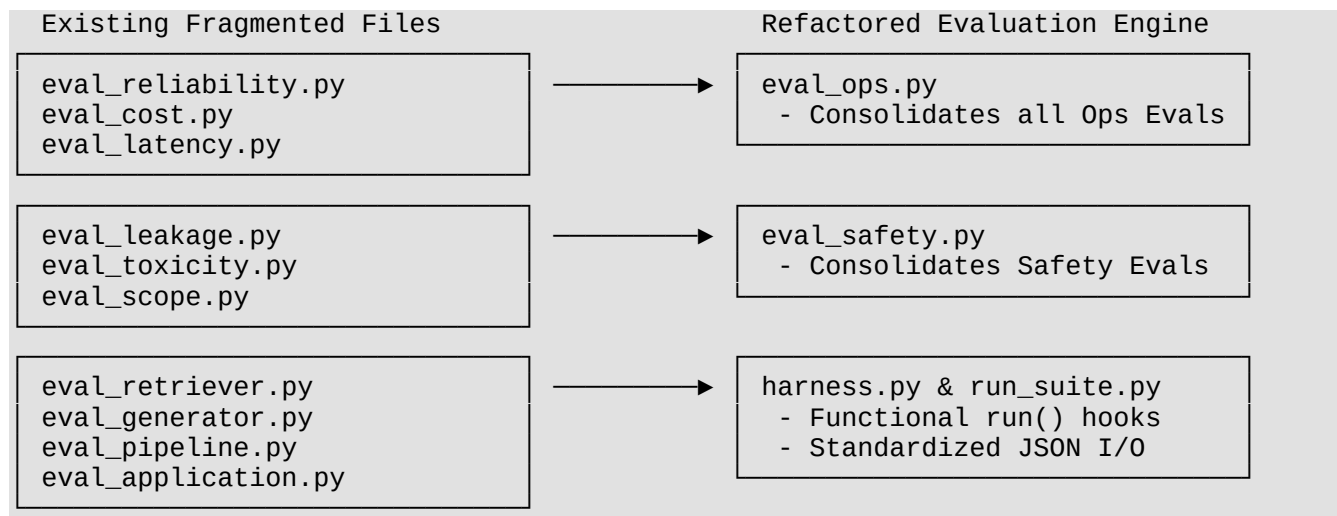
5.4 The Metric Registry Architecture (metric_registry.py)

To operationalize this logic, we store metric properties in `metric_registry.py`:

```
# Conceptual structure of metric_registry.py
METRIC_REGISTRY = {
    "recall": {
        "direction": "higher", # "higher" is better
        "noise_tolerance": 1.0 # 2 * sigma threshold
    },
    "contextual_relevancy": {
        "direction": "higher",
        "noise_tolerance": 2.5
    },
    "pii_leakage": {
        "direction": "lower", # "lower" is better
        "noise_tolerance": 5.0
    },
    "latency": {
        "direction": "lower",
        "noise_tolerance": 0.35
    }
}
```

6. Codebase Refactoring & Module Consolidation

To transform our existing evaluation code into an automated regression testing engine, we refactor our codebase:



6.1 Operational Evals Consolidation (eval_ops.py)

- We merge three independent operational evaluation files (`eval_reliability.py`,

`eval_cost.py`, and `eval_latency.py`) into a single unified script named `eval_ops.py`.

- Executing `eval_ops.py` runs Latency, Cost, and Reliability evaluations in sequence.

6.2 Safety Evals Consolidation (`eval_safety.py`)

- We merge three independent safety evaluation scripts (`eval_leakage.py`, `eval_toxicity.py`, and `eval_scope.py`) into a single script named `eval_safety.py`.
- Executing `eval_safety.py` measures Toxicity, PII Leakage, and Scope Adherence in sequence.

6.3 Quality Evals Refactoring & Harness Integration

- Our four quality evaluation files (`eval_retriever.py`, `eval_generator.py`, `eval_pipeline.py`, and `eval_application.py`) originally printed raw procedural output to the terminal.
- We refactor each file into modular Python functions by wrapping execution logic inside a standard `run()` entry-point and guarding standalone execution with `if __name__ == '__main__':` blocks.
- We introduce `harness.py` to standardize evaluation inputs and structure metric outputs into a consistent dictionary format across all quality evaluations.

6.4 Master Orchestrator (`run_suite.py`)

- We build `run_suite.py`, which serves as the primary orchestration entry point.
 - When executed, `run_suite.py` performs the following steps automatically:
 1. Invokes Quality Evals via `harness.py` (Retriever, Generator, Pipeline, Application).
 2. Invokes Safety Evals via `eval_safety.py`.
 3. Invokes Operational Evals via `eval_ops.py`.
 4. Collects all 14 metric scores and serializes them to `baseline.json` (for initial runs) or `candidate.json` (for subsequent experiment runs).
-

7. Practical Demonstration: Chunk Size Optimization Experiment

To demonstrate regression testing in practice, we conduct an experiment on our **CampusX Doubt Solver** RAG pipeline.

7.1 Hypothesis & Parameter Modification

- **Observation:** In our baseline system, **Contextual Relevancy** was underperforming at approximately **43% to 44%**.
- **Hypothesis:** We hypothesize that our text chunk size is too large (1500 characters), causing retrieved context to contain excess non relevant text. Reducing the chunk size will improve Contextual Relevancy.
- **Code Modification:** In `retriever.py`, we update the text splitter configuration:
 - Decreased `chunk_size` from **1500 to 500**.
 - Set `chunk_overlap` to **100**.
- **Database Reset:** Because chunking parameters changed, we wipe our existing Chroma vector database store and re-index all document embeddings from scratch.

7.2 Candidate Execution Log

- We execute our master test suite from the terminal, supplying a log message detailing our change:

```
python evals/run_suite.py --log "the chunk size is now 500 and the overlap is now 100"
```

- **Execution Flow:** `run_suite.py` executes 6 evaluation modules sequentially:
 1. **Retriever Evals:** Computes Recall (96%) and Precision (90%).
 2. **Generator Evals:** Computes Faithfulness and Answer Relevancy.
 3. **RAG Triad Evals:** Evaluates pipeline level interactions.
 4. **Application Evals:** Measures end-to-end task completion.
 5. **Safety Evals:** Computes Toxicity, PII Leakage, and Scope Adherence.
 6. **Operational Evals:** Computes Latency, Cost, and Reliability.
- Execution takes approximately **5 minutes** to complete and costs roughly **₹25 to ₹30** in LLM API queries for a single run over our small test dataset.

8. Differential Results Analysis & Decision Framework

8.1 Comparison Output (`compare.py --all`)

After candidate execution completes and writes to `candidate.json`, we run our comparison script:

```
python evals/compare.py --all
```

The script evaluates the deltas between `baseline.json` and `candidate.json` against

metric_registry.py, producing the following summary:

Evaluation Metric	Baseline Score	Candidate Score	Delta (\$\Delta\$)	Noise Tolerance (\$2\sigma\$)	Result Classification
Recall	High	96.0%	Within band	\$1.0\%\$	FLAT
Precision	High	90.0%	Within band	\$1.0\%\$	FLAT
Faithfulness	Standard	Standard	Within band	\$2.0\%\$	FLAT
Answer Relevancy	Standard	Standard	Within band	\$2.0\%\$	FLAT
4-5 Unnamed Metrics	Varied	Varied	Positive shift	Exceeds \$+2\sigma\$	IMPROVED
Contextual Relevancy	~45.0%	38.0%	-7.0%	\$2.5\%\$	REGRESSED
PII Leakage (Safety)	Baseline	Degraded	-20.0% (Score 6%)	\$5.0\%\$	REGRESSED

8.2 Analyzing System Regressions

- Contextual Relevancy Failed to Improve:** Despite reducing chunk size to boost Contextual Relevancy, the metric actually **regressed from ~45% down to 38%**.
- Severe Safety Metric Degradation:** PII Leakage suffered a severe **20% drop in safety score** (falling to a score of 6%). A 20% degradation in safety represents a critical red flag for production systems.

8.3 Automated Decision Engine (promote.py)

To evaluate whether a candidate version should move to deployment, we build a decision framework script named `promote.py`.

- Metric Tier Categorization:**
 - Tier 1 (Critical / Non Negotiable):** Safety metrics (e.g., PII Leakage, Toxicity). Any regression in Tier 1 triggers an automatic build block.
 - Tier 2 (Core Quality):** Key quality metrics (e.g., Contextual Relevancy, Faithfulness). Regression in Tier 2 triggers a human expert review requirement.
 - Tier 3 (Operational Trade-offs):** Minor latency or cost increases that can be tolerated if quality improves.
- Execution Outcome:** Running `python evals/promote.py` evaluates the output of `compare.py` against our tier rules:

[Decision Framework Output]

STATUS: BLOCK

REASON: Critical Tier 1 Safety Metric (PII Leakage) regressed by 20%.

DEPLOYMENT HALTED. Revert candidate changes.

- **Verdict:** Because a Tier 1 safety metric regressed by 20%, the deployment is automatically blocked. If only Contextual Relevancy had dropped while 5 other metrics improved, `promote.py` would have assigned a REVIEW status for human domain experts to evaluate trade-offs.
-

9. Advanced Industry Practices & Technical Q&A

9.1 Multi Dimensional Evaluation vs. Composite Scoring

- *Question:* Can we collapse all 14 metrics into a single weighted composite score (e.g., using a weighted average formula) to judge overall system quality?
- *Answer:* While creating a single composite score is a tempting simplification, it is generally **discouraged in real world RAG engineering**. Distinct metrics exist because they isolate failures to specific pipeline stages (e.g., distinguishing whether a bad answer stems from retrieval failure vs. generator hallucination). Merging metrics into a single number masks critical localized regressions such as a major safety violation being hidden by minor gains in answer length.

9.2 Diagnosing Pipeline Failures (Retriever vs. Generator)

- *Question:* How do we diagnose whether an incorrect final answer is caused by retrieval failure or generator failure?
- *Answer:* We isolate failure points by analyzing complementary metric pairs:
 - If **Recall**, **Precision**, and **Contextual Relevancy** are high but the final answer is incorrect, the failure lies in the **Generator**.
 - If generator metrics are solid but retrieval metrics are depressed, the failure lies in the **Retriever**.

9.3 Local CI vs. Production CI/CD Integration

- *Question:* Does running this test suite locally mimic Continuous Integration (CI)?
- *Answer:* Yes. Executing `run_suite.py` locally acts as local CI testing. In production setups, this entire Python pipeline is integrated into automated CI/CD runners (such as **GitHub Actions**). Every pull request automatically triggers `run_suite.py` and `compare.py` to gate deployments before merging code.

9.4 Statistical Rigor at Enterprise Scale

- *Question:* How do high stakes production systems (e.g., ChatGPT) handle evaluation

uncertainty at scale?

- *Answer:* When millions of dollars are at stake, enterprise evaluation setups move beyond simple standard deviation thresholds (2σ) . They incorporate formal statistical hypothesis testing evaluating **p values** and **statistical significance** to rigorously verify that measured metric deltas represent genuine performance shifts rather than random sampling noise.